

twyla

A static-site generator whose only authoring language is typst.

Twyla embeds the typst compiler as a Rust library (the typst crate), not the CLI, so it can route typst files to URLs, intercept asset resolution to fingerprint/optimize images, walk the compiled document for metadata, emit RSS/Atom, and (eventually) do incremental rebuilds.

The driving target is porting samsartor.com (currently zola + tera + markdown, at ~/Src/site) to twyla + typst, with HTML AST equivalence as the porting harness.

Why pure typst (no template layer)

Considered: a tera-like template engine layered on top of typst, by analogy with zola layering tera on top of markdown. Rejected — typst already provides the moving parts a template language would re-invent:

- **Inheritance** — `base.html + {% block %}` becomes a typst function taking a body parameter, with `show/set` rules supplying the defaults.
- **Filters** — typst functions compose better than pipeline syntax.
- **Shortcodes** — already just typst functions, no separate registry.
- **HTML output** — `html.elem(..)` and the typed `html.div(..)` family let us emit arbitrary tag + class + attribute structures.

Cost of **adding** a template language: two scoping models, two escape hatches, two contexts for the author to switch between. Cost of **not** adding one: typst's HTML export is younger than tera, so ergonomic rough edges land on us. That's where twyla earns its keep — wrapping the crate gives us room to shape output rather than living with defaults.

Architecture

Routing & bundle

`#document(path, ..)` (typst PR 7964) is the routing primitive: one call per emitted HTML file. Cross-document links use labels; typst computes relative paths automatically. `#asset(path, bytes)` writes raw files into the same bundle output.

Every compile goes through a synthesized virtual `main.typ` — never on disk, just an in-memory Source pre-populated in the World cache at `/__twyla_main.typ`. Twyla scans `cwd/content/*.typ`, derives a slug per file from its stem (`foo.typ` → `/foo/`), and emits one `#document(..)` call per slug. The single-page case (`render_slug`, used by `check`, `render`, and `dev-server` per-request compile) is the same machinery with one entry.

Per-document routing context is carried via a labelled metadata marker. The generated main inserts `#metadata(url-path: "foo")`

`<twyla-page>` inside each document body; page-template reads its own slug with context `query(<twyla-page>.first().value.url-path`. The query is **per-document scope** — each routed output sees only its own marker, verified by `render_site_smoke` in tests. This is the load-bearing pattern; everything else that needs per-document data from twyla into typst (title for `<head>`, date for feed entries, etc.) will reuse the same shape.

Bundle output paths **are** URLs: `#document("foo/index.html")` lands a file at `foo/index.html` in the build output. The dev server reads `bundle.files`; twyla build writes them under `output_dir/`.

Single-document files are **not** valid entrypoints anymore — content files no longer call `#document(..)` themselves (the template did, previously). Twyla owns routing now.

Underscore-prefixed names are reserved for special routes: `_index.typ` (home / section index) and drafts. Today everything `_*` is skipped; home-page wiring is the next revision.

Asset model

Two orthogonal axes for asset primitives — **input source** and **output kind**:

	Output: bytes	Output: url
Input: path	<code>read(path) -> bytes</code> <i>Already typst's built-in.</i>	<code>asset-url(path) -> str</code> — emit URL pointing at file. Eventually fingerprinted.
Input: bytes	<i>byte transforms (font rewrite, webp encode); rust or typst.</i>	<code>content-url(bytes, name) -> str</code> — hash, write to bundle, URL.
Input: content (<i>typst Content</i>)	<code>render(content, format) -> bytes</code> — sub-compile. Overlaps with typst's <code>#document(path,...)</code> .	<code>asset.from-content(...).url</code> — render + register + URL.

Principle: byte transforms run in Rust (or in user typst where ergonomic); presentation decisions (inline `<svg>` vs `<img>` vs `<picture>` vs `<use href>` vs `iframe`) run in user typst templates. URL-returning primitives are the seam: above them typst owns layout and emission; below them Rust owns paths, fingerprinting, and the on-disk bundle.

Phase 1 implements only `read` (free from typst). URL primitives are deferred until a second use case forces the API surface.

`#document` is the page-level expression of “content input → URL.” Open question whether the eventual `asset.from-content(..)` API subsumes it, or whether they stay separate (pages routed by content layout vs assets routed by hash).

Resolution pass

Twyla owns a post-compile **resolution pass** — a walk over the bundle's serialized output that finds placeholder markers and substitutes their final form. Inspired by the future/promise pattern: typst emits a deferred value; Rust resolves it later. This is the seam between “things typst can express” and “things we want in the output.”

Phase 1 has one placeholder: **raw HTML**. Typst has no `html.raw`; only `<script>` and `<style>` bodies are emitted unescaped (verified: `typst-html::tag::is_raw`). We exploit that: user typst calls `raw-html(content)` which emits `<script type="x-twyla-raw-html">..bytes..</script>`; the resolution pass replaces each match with the inner content spliced inline. Used by the `svg/diagram` shortcodes to embed external SVG files.

Anticipated placeholders, each earned by a use case:

- **Image optimization** — `<twyla-image data-src=".." data-sizes="..">` resolves to `<picture>/<img srcset>` after webp/avif encoding.
- **Asset-as-content** — sub-compile registered typst content to SVG, splice inline.
- **Feed expansion** — `<twyla-feed-entries data-query="..">` resolves to N siblings driven by introspector query results.

Implementation strategy: keep the `str-replace` pattern until we have ≥ 2 marker kinds, then generalize to a typed handler registry.

The resolution pass is **the** reason twyla doesn't need to fork typst — it's pure post-processing of typst output, no internals touched.

Tooling

Single twyla binary, clap-driven. Two audiences.

End-user commands (turn-key, cwd-driven)

- **twyla serve** — dev server on `127.0.0.1:1111`. Zero flags. Routes `/<slug>/` to `render_slug`, falls back to `cwd/static/` then `cwd/content/` (zola colocated-asset convention) for everything else. /

is a placeholder index until the home page is wired. Single-threaded accept loop, hand-rolled HTTP/1.1 over std::net — no async, no extra deps. No watch / no HMR yet.

- **twyla build [-o <dir>]** — compile every page, write <dir>/<slug>/index.html per routed doc, copy static/ verbatim and content/*.!{typ,!md} (the asset bridge). Default output ./public/, matching zola. Existing files are overwritten in place; nothing is removed. Will conflict with zola's public/ in the site repo today — pass -o /tmp/twyla-test/ to keep them separate until twyla has its own sass/JS story.

Porting harness (flag-driven)

- **twyla render [--site-root <dir>] <slug>** — compile one slug as a single-document bundle, run resolution pass, print HTML. Debug tool; same code path as the dev server's per-request compile.
- **twyla check [--site-root <dir>] <slug>** — render + full-page diff for a ported page. Inputs inferred from zola's layout: content/<slug>.typ vs public/<slug>/index.html. Relaxations are the cumulative universal set found across ported pages so far (textonly-pre, ignore-attr td/th:style); no per-page config yet.
- **twyla diff [--textonly-pre] [--ignore-attr <tag>:<attr>]... <expected> <actual>** — structural AST diff with optional porting relaxations.
- **twyla import <md>** — best-effort md→typ draft generator on stdout. Pulldown-cmark walk plus a shortcode pre/postprocess pass; handles the common shape of the personal-site corpus. Raw HTML and unusual shortcodes get a // TODO twyla-import: ... comment.

The render pipeline is exposed as library functions (twyla::render::{render_site, render_slug}) so check, serve, and build all call it in-process. No shell glue, no rebuild step. Earlier iterations shipped three separate bins (twyla-render-page, twyla-diff, twyla-extract) plus a port-page.sh driver; all consolidated.

check stays zola-specific by design; the eventual EXAMPLES-corpus verifier may want a manifest, but with one site the directory layout **is** the manifest. Revisit when ≥ 5 pages need page-specific overrides or when adding a second corpus.

AST diff harness (twyla::diff)

Library + CLI. Parses both sides via html5ever, normalizes (lowercase tags/attrs, sort attrs and class tokens, collapse inter-block whitespace, preserve <pre>/<code>/<script>/<style>/<textarea> text verbatim, drop comments), walks both trees in lockstep, reports the first structural divergence.

Two parser quirks applied symmetrically to both sides — not opt-in relaxations, since they're tool artifacts on the typst side that correspond to no authorial intent on either:

- <noscript> content is parsed as HTML, not text. html5ever defaults to "scripting on," which turns noscript bodies into text nodes; we flip scripting_enabled: false. Otherwise zola's pretty-formatted <link> siblings diverge from typst's tightly-packed ones.
- Pure-whitespace text inside <script>/<style> is dropped. Typst's pretty-printer wraps even an empty <script src="..."> body in a newline + indent; zola emits <script src="..."></script>. Significant text inside the same tags is left untouched. Doesn't apply to <pre>/ <code>/ <textarea>, where inter-token whitespace inside highlighted code is meaningful.

Relaxations are opt-in (Matcher, RelaxationRule) pairs, first-match wins. Defaults to zero — porting starts strict.

Matcher	Matches
Tag("p")	every <p> element
TagAttr { tag, attr, value }	matching tag with attr == value
TagAttrExists { tag, attr }	matching tag with attr set

RelaxationRule	Effect
IgnoreEntirely	subtree considered equal regardless
IgnoreAttribute(name)	skip one attribute on matched element
TextOnly	compare concatenated text only; ignore structure

Standing rule: every new relaxation is a place twyla and zola diverge. Run them by Sam first.

Deliberately deferred: per-section relaxations (different rules under different ancestors), multi-divergence reporting (today: bails on first divergence), attribute-value matchers (“ignore style only when value = color: red”).

Status

1. ~~HTML-export-prototype~~ — done (experiment 1).
2. ~~Multi-document-bundle-output~~ — done (experiment 2).
3. ~~AST-diff-harness~~ — done (twyla::diff).
4. ~~Port one page, body only (guis-2)~~ — done. Two relaxations: `--textonly-pre` (code blocks) and `--ignore-attr td:style --ignore-attr th:style` (typst tables don’t emit alignment as CSS).
5. ~~Port guis-2 full page~~ — done. `templates/page.typ` in the site repo ports `base.html + page.html + components.html::mainpills`; `guis-2.typ` opens with `#show: page-template.with(url-path, title, description, date, ...)`. Same two relaxations as the body milestone.
6. ~~CLI-consolidation~~ — done. Single clap-driven twyla binary with `render/diff/port` subcommands; render pipeline exposed as a library so port calls it in-process. Old triple-bin + shell driver removed.
7. ~~Port guis-3~~ — done. port generalized to check `<slug>` (zola layout is the manifest; relaxations are universal defaults). Two new porting hazards surfaced (below): typst paragraph-wrap of bare inline elements, and zola’s anchor-only-link absolutization.
8. ~~Port guis-1 via twyla import~~ — done. Import generated a near-complete draft; two iterations on the import binary itself (centered-block trailing whitespace causing nested `<p>s`, ordered-list `<ol>` vs `<ul>` from `Tag::List(Some(_))`) then a two-line manual fixup (`_page-url + url-path` placeholders) got it to match zola.
9. ~~Multi-entriypoint-compile + url-path-scope-shift~~ — done. Every compile goes through a synthesized virtual `main.typ` that emits one `#document(..)` per `content/*.typ` and a per-document `<twyla-page>` metadata marker. `page-template` no longer takes a `url-path` parameter and no longer calls `#document(..)` itself; it reads its slug from the bundle introspector via context query(`<twyla-page>`). `cmd_render/cmd_check` switched from `path` to `slug`. Verified end-to-end (`render_site_smoke`): compiling all three `guis-*` pages in one bundle, no label-scope leak between docs.
10. ~~Dev-server (twyla serve)~~ — done. Zero-flag, `127.0.0.1:1111`, hand-rolled HTTP/1.1 over `std::net`, no extra deps. Compiles per request (3s debug, sub-second `--release`, ms with the shared-world revision still queued). Static fallback is `cwd/static/` then `cwd/content/` (mirrors zola’s colocated assets). `/` is a placeholder index until the home page is ported.
11. ~~twyla-build~~ — done. Walks `render_site` output, writes `<output>/<slug>/index.html`, copies `static/` verbatim and `content/` minus `.typ/.md`. Default `./public/`, override via `-o`. Verified: `twyla build -o /tmp/twyla-test/` produces output whose `guis-2/index.html` matches zola through the porting diff.

Roadmap

Working assumption: walk from “one page, body” outward, adding twyla infrastructure only when a porting case forces it.

Near term

1. ~~twyla-import <md>~~ — done. Pulldown-cmark walk + a shortcode pre/postprocess pass (HTML-comment markers around `{% ... %}` / `{{ ... }}` so pulldown emits them as discrete HTML events). Coverage: frontmatter → page-template boilerplate, ATX headings → h1/h2 helpers + slug, paragraphs, blockquotes, ATX rules, lists (ordered + unordered), fenced/inline code, links (external + anchor + root-relative), emphasis/strong, the four ported shortcodes (centered/svg/image/diagram). Unknown tags/shortcodes get a `// TODO twyla-import: ...` flag. Two placeholders the porter fills (`_page-url`, `url-path`).
2. **Open: who applies the page template?** Today `guis-2.typ` / `guis-3.typ` open with `#show: page-template.with(..)` — explicit, one line of boilerplate per page. Alternative: twyla's eventual generated `main.typ` auto-applies a template per content file (less magic visible to the author). Defer until enough pages exist that the boilerplate hurts.
3. **Open: link show rule home.** — done. The external-link + anchor-link rule lives in page-template, which derives `_page-url` from `url-path` (strip `.html`, prepend `_base-url`, append `/`). Same scope grew to cover `set smartquote(enabled: true)` and the heading auto-slugifier (`show heading: it => html.elem("h" + level, attrs: (id: slug), it.body)` with a small `_text-of` + `_slugify` pair in the template). Net: a ported page's prelude is now three lines (two imports + `#show: page-template.with(..)`); = Heading syntax replaces `#h1("slug")` helpers.

Medium term — twyla becomes an SSG

Queued, in the order we plan to land them:

1. **Home page (`content/_index.typ` → `/`).** First special-route. Two knock-on concerns: scan must stop skipping `_index.typ` and route it to the bundle root; the index needs cross-doc post data (title, date, description, slug) which means the virtual `main.typ` writes one `<twyla-post>`-labelled metadata block per document with those fields, and `_index.typ` queries them. Same per-document-scope shape as `<twyla-page>`; first second-marker for the resolution registry argument.
2. **Shared world + comemo carry + dependencies() invalidation.** One `RenderWorld` lives behind a `Mutex` across requests; the `typst_kit::watcher::Watcher` thread waits on `World::dependencies()` changes and runs `world.reset()` + `comemo::evict(10)` on each batched event. Single-doc compiles become fast (ms) on warm cache. No browser-side reload yet — manual refresh after the watcher prints a recompile line. Mirrors typst's own CLI watch loop (`crates/typst-cli/src/watch.rs`, `typst_kit::watcher`) — same crate, same pattern, no fork.
3. **WS-driven hot reload.** Server injects a tiny `<script>` into served HTML that opens a websocket; watcher revision signals "recompiled, reload" over it. Clean split from the shared-World revision — rollback story stays simple if the WS layer misbehaves.
4. **Real asset-url primitive.** Drop the hardcoded `base_url` in `templates/shortcodes.typ`. Comes from a twyla config layer (TOML or typst-side). This is the moment we promote our one resolution-pass placeholder into a registry, because the URL primitive needs hooks for fingerprinting/dedup.
5. **Sass / JS bundling.** Today twyla's dev workflow detours back through zola (`zola build --drafts` for `public/site.css`, `yarn build` for `public/scripts/*.js`); `static/` holds symlinks pointing at those outputs. Real fix: twyla shells out to sass and vite/esbuild, or just dart-sass as a library, and owns these directly. Until then the symlink hack is the bridge.
6. **Image optimization pipeline.** Webp/avif encoding, srcset generation, in Rust as a resolution-pass handler. User typst calls one function; the registered handler does the byte transforms and emits the resolved `<picture>`.
7. **Feed generation.** `#document("atom.xml", ..)` in twyla's library, querying `bundle.introspector` for kind: "post" entries. Reuses the same `<twyla-post>` metadata the home page reads.

Longer term

1. **Port more pages.** `coroutines-1` / `coroutines-2` (similar shape to `guis-*`), `matfusion` (first paper page — exercises `_mainpills` which is carried but untested), `what-is-color/` (first nested section, exercises subdirectory routing), `dissertation`, `self-referential`, etc. Each is a use case for `twyla` import followed by `twyla` check iteration.
2. **Twyla config layer (`twyla.toml`).** `base_url`, `deploy` overrides. All keys optional — empty file is still turn-key. Probably forced by the `asset-url` primitive.

Validation

1. **EXAMPLES-corpus test.** Run a port-page-style diff against representative pages from [getzola's EXAMPLES](#). Catches feature breadth the personal site doesn't exercise. Should produce a corpus of (`zola-html`, `twyla-html`, `relaxations`) triples that we keep green on CI.

Lessons (from porting `guis-1` + `guis-2` + `guis-3`)

Compact gotcha list; details in commits.

- **Smart quotes** — leave `typst`'s `smartquote` on; matches `zola`'s `smart_punctuation = true` character-for-character.
- **Show rules fire in HTML mode** (verified `de6f400`). Folk claim otherwise; not true for content-transformation rules. Layout-specific rules (positioning, frame production) may not, untested.
- **Pulldown auto-wraps inline content in `<p>` inside containers** (`<div>`, `<blockquote>`, `<td>`); `typst`'s HTML export doesn't. Wrap manually with `#html.p[...]` until we find a cleaner pattern.
- **Native `#table` works for HTML.** Only divergence: `align: (..)` doesn't reflect into per-cell `style="text-align:..."` (`typst` treats it as layout-only). Relax `--ignore-attr td:style` `--ignore-attr th:style` to absorb.
- **Show rules can't replace `typst`'s auto-generated wrapper.** A `show table.cell.where(x: 0): it => html.elem("td", ..)` produces `<td><td ..>..</td></td>`. Same hazard expected on `list.item`, `enum.item`, etc.
- **`html.a(rel: ..)` takes an array of enum tokens, not a space-joined string:** `rel: ("noopener", "external")`.
- **External-link auto-attrs** (`rel="noopener external"` `target="_blank"`) via `show link: it => ...` checking `type(it.dest) == str` and `prefix`.
- **`html.elem` for non-typed attrs.** `html.div` doesn't expose `align`; use `html.elem("div", attrs: (align: "center"), ..)`.
- **Inline SVG via the resolution pass.** `<script type="x-twyla-raw-html">` falls into `RawMode::Keep` because the `type` attribute isn't `text/javascript`.
- **`Typst #let` chain-across-newlines hazard.** `#let f(s) = s\n .replace(..)` parses as `identity-of-s` with the `dot-calls` as orphan markup, no error. Wrap multi-line chains in `(...)`.
- **Bypass `typst`'s auto-`<head>/<body>` by emitting your own `<html>`.** `finalize_dom (typst-html/ document.rs:252)` short-circuits when the single top-level element is `<html>` — uses ours verbatim, skipping the default head. Cost: footnotes are unsupported in that mode.
- **Reserved-word attrs need string keys.** `attrs: (as: "style", ...)` is a parse error; `as` is reserved, `type` is a builtin. Quote them: `("as": "style", "type": "...")`.
- **`[#]` in markup is a parse error.** `#` starts a code expression, so `[#]` opens an expression with no content. Escape as `[\\#]` to emit a literal hash character (used in the `description_hash` span).
- **Pulldown auto-slugifies headings; `typst`'s = doesn't.** For `zola` parity each section heading needs an explicit id. Solved by a `show heading` rule in `page-template` that emits `<h{level} id="<slug>">..</h{level}>`. With that, `plain = Heading / ==`
Heading markup matches `zola`'s output. Two more notes from the cleanup: in HTML mode `typst-html` shifts `=` to `<h2>` by default (the document title takes `<h1>`); a `show` rule using `it.level` bypasses that. And `labels: = Foo <bar>` sets `id="bar"` and `#link(<bar>)[anchor text]` works without `set heading(numbering:`

..) — @bar is the form that needs numbering (for the auto-generated reference text). Labels could replace text-slugs for cross-doc references once the corpus has drifted from zola; for now, matching zola slugs is what we need.

- **Inline `<br>` is auto-paragraph-wrapped.** `#html.br()` on its own line produces `<p><br></p>` — typst treats `<br>` as inline content. `<hr>` is block, so `#html.hr()` doesn't wrap. For a bare sibling `<br>` (zola/pulldown emits this between adjacent block shortcodes), splice via `raw-html("<br>")`.
- **Anchor-wrapping-block + soft-break + empty `<a>`.** Pulldown can't nest a block inside an inline `<a>`, so `<a>[fenced code]</a>` in MD emits broken HTML (`<p>text\n<a></p><pre>..</pre></a>`) that `html5ever` then adoption-agencies into "empty `<a>` trapped in the prior `<p>`, sibling `<a>` reconstructed around the `<pre>`". Two typst hazards block clean reproduction: `#html.a()` outside a paragraph gets `<p>`-wrapped, and a soft break before any inline element injects a `<span style="white-space: pre-wrap">` to preserve whitespace. Easiest fix: splice both halves via `raw-html`.
- **Zola absolutizes anchor-only links.** `[text](#frag)` in MD becomes `<a href="<base_url>/<slug>/#frag">` in zola output, not `href="#frag"`. Extend the link show rule to prepend a per-page `_page-url`. Hardcoded for now; lift into `page.typ` once `twyla` has a config layer.
- **Virtual `main.typ` is dead-cheap to inject.** `RenderWorld::source()` consults an in-memory `HashMap<FileId, Source>` before reading from disk; pre-populate that map with the synthesized main keyed at `/__twyla_main.typ` (any `vpath` that's not a real file works). No custom `World` trait impl changes beyond this; typst's bundle compile doesn't care that the entrypoint isn't on disk.
- **Per-document `<label>` query is the routing-context primitive.** `#document(path)[..]` scopes `query(<label>)` to the document body, so each routed output sees only its own marker — exactly what per-page-config wants. Markup-mode label syntax (`#metadata(..) <label>`) is the one that parses; emit your main with `[..]` content blocks rather than code-mode `{..}` blocks when labelling.
- **`bundle.files` is `Arc<IndexMap<VirtualPath, BundleFile>>`.** Iterate with `.iter()` (for `x` in `&bundle.files` won't auto-deref through `Arc`). Each entry is a `BundleFile` enum; match `BundleFile::Document(BundleDocument::Html(doc))` for HTML pages. `path.get_without_slash()` gives the slash-relative `&str` that matches the `#document(..)` path string.
- **Show rules from `#included` files stay scoped to that include's body.** A multi-document virtual main can include `N` content files whose `#show: page-template.with(..)` rules don't leak between documents. Verified by compiling `guis-{1,2,3}` in one bundle and diffing each against zola (`render_site_smoke`).
- **Page-template doesn't call `document()` when `twyla` owns routing.** Returning the `html.elem("html", ..)` content from page-template is enough; the generated main's `#document(path, body)` wraps it. Title/description args on `#document(..)` are typst-level metadata, not HTML — we set the HTML `<title>` ourselves and the `document-arg` metadata is unused today (will matter for feeds).
- **context blocks compose inside show-rule transformers.** The anchor-link branch of the link show rule wraps `html.a(href: ..)` in `context { let slug = query(<twyla-page>)...; .. }` — the lazy evaluation point per link is acceptable (no perf issue at our scale) and avoids restructuring page-template around top-level context.

Upstream-watch list

Worth raising upstream (or watching for) on typst:

1. `html.raw(content)` — first-class verbatim HTML. Today the resolution pass abuses `<script>`. Likely controversial (typst values structured output) but every HTML consumer hits this.
2. `#table(aligned: ..)` reflecting into `style="text-align: .."` on `<th>/<td>` in HTML mode. Same for `colspan/rowspan` attributes.
3. Show rules on `table.cell/list.item/enum.item` that can **replace** typst's auto-generated wrapper in HTML, not just decorate its body.

4. Hook on `image()` to emit external `src` instead of data URLs in HTML mode — would benefit anyone doing HTML export at scale.
5. Per-document defaults override (e.g. document-level `smartquote(enabled: false)` declared once).
6. Introspector exposure of cross-document link resolution data — useful for fingerprint-then-rewrite passes.
7. Opt-out for the paragraph auto-wrap of bare inline elements (`<br>`, empty `<a>`, etc.) when they appear at block context. Today the only escape is to splice via `raw-html` — fine as a workaround, ugly when the motivation is just “don’t put this in a `<p>`.”
8. Suppress the `<span style="white-space: pre-wrap">` whitespace shim in HTML output. Useful for SVG/serif typography, but for porting parity with non-typst HTML it’s pure noise.

Fork vs library

Standing assumption: **do not fork typst**. Twyla is a Rust binary that consumes typst as a library, supplies a custom World, ships a small typst library of helpers, and post-processes the bundle. If something is genuinely better as an upstream change, **upstream it** before forking. Fork threshold: library + upstreaming both proven inadequate.

Why: typst’s HTML/bundle work is actively evolving in main. A fork puts us permanently downstream of grammar, library, and HTML-output changes — a real maintenance tax paid forever for benefits mostly reachable from outside.

Decisions to date (every feature considered has landed on “library”): asset fingerprinting (post-export DOM walk), image optimization (twyla function bypassing `image()`), pretty URLs (`#document(..)` paths), path computation, hot reload (WebSocket-inject during dev), RSS/sitemap, per-document `smartquote/lang` (already configurable), incremental compile (comemo), file mtime via `sys.inputs`, `async/HTTP` at compile (pre-fetch in Rust). No fork: HTML-syntax-mode (the JSX-style sugar isn’t worth shadowing typst’s grammar).

If we ever hit a real wall, document what we tried and why it didn’t work — that’s the evidence base for reconsidering.

Notes

- VCS: both `~/Src/twyla` and `~/Src/site` are managed with `jj`. **Rule:** always `jj new` before editing — `jj` has no staging area, so edits land in whatever revision the working copy points at.
- Typst pinned to main `de6f400` (2026-04-11) for the bundle export feature (PR 7964). Switch back to `crates.io` once `0.15.x` ships.
- Tera shortcodes in scope for porting: `centered`, `diagram`, `image`, `math`, `svg`, `tagline`, `var` (`~/Src/site/templates/shortcodes/`). Phase 1 covered the first four; `math/tagline/var` come with pages that use them.
- Tera templates in scope: `base.html`, `page.html`, `section.html`, `index.html`, `components.html`, `404.html`, `atom.xml`, `paperpills.html`. Phase 1 covered `base.html + page.html + components.html`:`mainpills` (the last untested until a paper page).
- Known porting hazards: ROT13-encoded email obfuscation in footer, conditional asset loading (`page.extra.tilings`), SVG inlining with font-family rewriting (handled by the `svg/diagram` shortcodes).
- Dev-asset bootstrap (until twyla owns sass/JS): `~/Src/site/static/` holds symlinks pointing at zola/vite outputs in `~/Src/site/public/` — `static/site.css -> ../public/site.css`, `static/scripts/{earllysite,site,style}.js,css -> ../../public/scripts/...`. Recompile CSS by re-running `zola build --drafts` (or `sass sass/site.sass public/site.css` directly if `sass` is on PATH); JS via `yarn build`. `static/scripts/` is in `.gitignore` so only `static/site.css` is tracked.
- `twyla serve` recompiles the requested page per request — `3s debug`, sub-second `--release`. Shared-World + `comemo` carry is the queued revision that fixes this; until then prefer `cargo build --release && ./target/release/twyla serve` for iteration.